SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.
Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.
Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I B.Maheshwar (192411195) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.Maheshwar

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

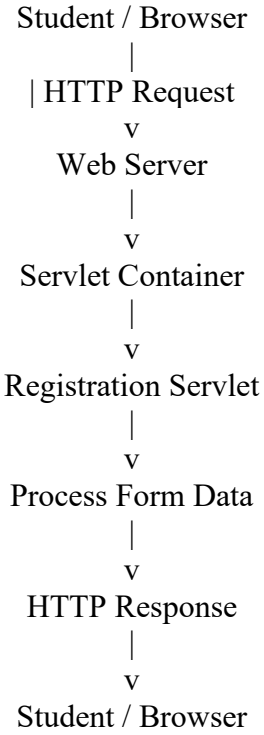
CSA4305 – INTERNET PROGRAMMING
ASSESSMENT TOOL 1 – SCENARIO-BASED ASSESSMENT

Question 1 – Servlet-Based Student Registration System

A college wants an online student registration system in which students submit their details through a web form and a Java Servlet processes the data.

1. Servlet Architecture

The servlet-based application follows a client-server architecture:



Client/Browser: Student enters registration details.

Web Server: Receives HTTP requests from the browser.

Servlet Container: Creates and manages the servlet.

Servlet: Reads form data, processes it, and generates the response.

Response: The processed result is returned to the browser.

2. HTML Student Registration Form

```
<!DOCTYPE html>
<html>
<head>
<title>Student Registration</title>
</head>
<body>
<h2>Student Registration</h2>
```

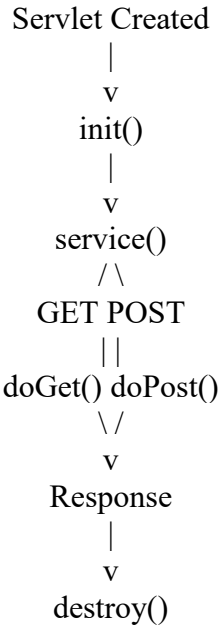
```
<form action="register" method="post">
Name: <input type="text" name="name"><br><br>
Email: <input type="email" name="email"><br><br>
Course: <input type="text" name="course"><br><br>
Age: <input type="number" name="age"><br><br>
<input type="submit" value="Register">
</form>

</body>
</html>
```

3. Servlet Implementation Using GET and POST

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/register")
public class StudentRegistrationServlet extends HttpServlet {

public void init() throws ServletException {
System.out.println("Servlet Initialized");
}

protected void doGet(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {

response.setContentType("text/html");
PrintWriter out = response.getWriter();

String name = request.getParameter("name");
String email = request.getParameter("email");
String course = request.getParameter("course");

out.println("<h2>Registration Details - GET</h2>");
out.println("Name: " + name + "<br>");
out.println("Email: " + email + "<br>");
out.println("Course: " + course);
}

protected void doPost(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {

response.setContentType("text/html");
PrintWriter out = response.getWriter();
```

```
String name = request.getParameter("name");
String email = request.getParameter("email");
String course = request.getParameter("course");

out.println("<h2>Registration Successful</h2>");
out.println("Name: " + name + "<br>");
out.println("Email: " + email + "<br>");
out.println("Course: " + course + "<br>");
}

public void destroy() {
System.out.println("Servlet Destroyed");
}
}
```

4. Servlet Life Cycle



init(): Called once when the servlet is created. It is used for initialization.

service(): Called for each client request and dispatches the request to doGet() or doPost().

destroy(): Called before the servlet is removed from service. It is used to release resources.

5. GET and POST Comparison

Feature	GET	POST
Data location	Sent in URL	Sent in request body
Visibility	Can appear in URL/history	Not displayed in URL
Use	Retrieving data	Submitting data
Data size	Limited by URL length	Can send larger data
Sensitive data	Not suitable	More appropriate

6. Which Method is Better for Sensitive Data?

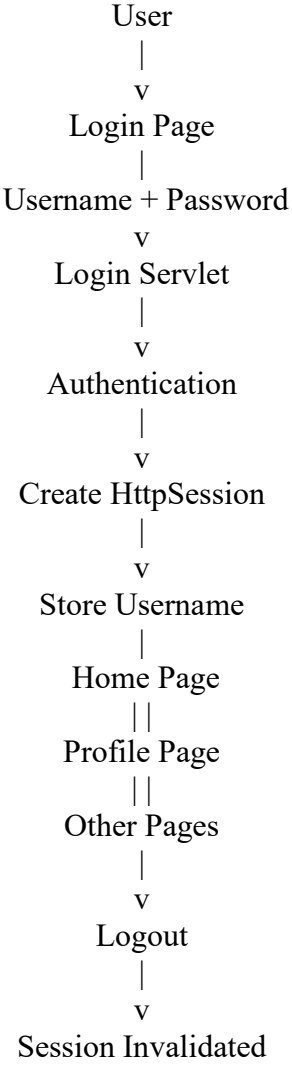
POST is more appropriate for sensitive registration data because the submitted values are placed in the HTTP request body instead of the URL. GET exposes parameters in the URL, where they may appear in browser history or logs. For real security, HTTPS must also be used because POST alone does not encrypt data.

Conclusion: The servlet receives student registration data, processes GET and POST requests, and returns a response. The servlet life cycle is managed by the servlet container.

Question 2 – Session Management Using HttpSession and Cookies

A website needs users to log in and maintain their session across multiple pages after authentication.

1. Session Management Design



2. Login Form

```
<form action="login" method="post">
Username: <input type="text" name="username"><br><br>
Password: <input type="password" name="password"><br><br>
```

```
<input type="submit" value="Login">
</form>
```

3. Login Servlet Using HttpSession

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

protected void doPost(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {

String username = request.getParameter("username");
String password = request.getParameter("password");

response.setContentType("text/html");
PrintWriter out = response.getWriter();

if (username.equals("admin") && password.equals("1234")) {

HttpSession session = request.getSession();
session.setAttribute("username", username);

out.println("<h2>Login Successful</h2>");
out.println("Welcome " + username);
out.println("<br><a href='home'>Go to Home</a>");

} else {
out.println("<h2>Invalid Username or Password</h2>");
}
}
}
```

4. Maintaining the Session Across Multiple Pages

```
HttpSession session = request.getSession(false);

if (session != null &&
session.getAttribute("username") != null) {

String username =
(String) session.getAttribute("username");

out.println("Welcome " + username);
out.println("You are logged in.");

} else {
```

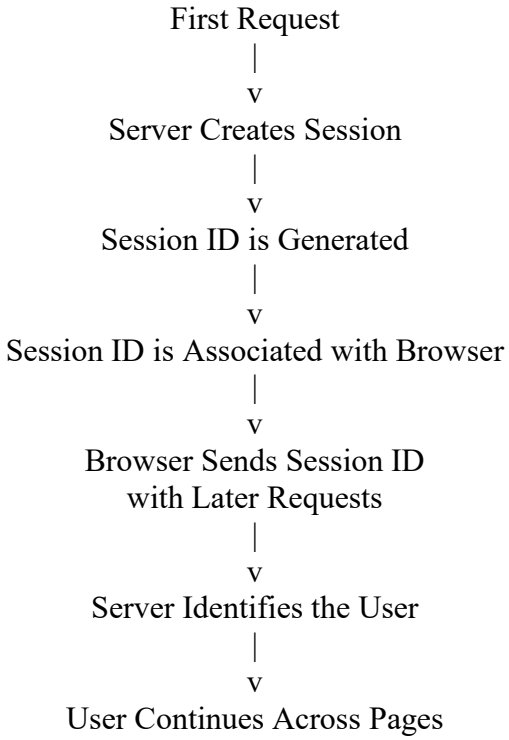
```
out.println("Please Login First");
}
```

5. Cookies in Servlets

A cookie is a small piece of information stored in the user's browser. It can be used for preferences or to help maintain a user's session.

```
// Create a cookie
Cookie cookie = new Cookie("username", username);
cookie.setMaxAge(60 * 60);
response.addCookie(cookie);

// Read cookies
Cookie[] cookies = request.getCookies();

if (cookies != null) {
for (Cookie cookie : cookies) {
if (cookie.getName().equals("username")) {
String username = cookie.getValue();
System.out.println(username);
}
}
}
```

6. How Session Tracking Works



In a typical servlet application, the session identifier can be maintained using a cookie such as JSESSIONID. The server uses the identifier to find the corresponding HttpSession.

7. HttpSession vs Cookies

Feature	HttpSession	Cookies

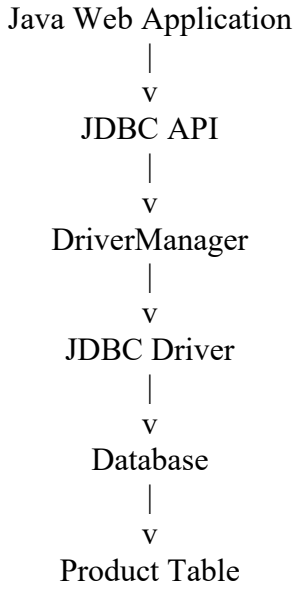
Storage	Session data is maintained on server	Data is stored in browser
Security	Important data can remain server-side	Data is available to client
Capacity	Can store multiple session attributes	Suitable for small data
Use	Login/authentication state	Preferences and small values
Lifetime	Ends on timeout/invalidation	Controlled using cookie expiry

8. Decision
For an authenticated website, HttpSession should be used as the main session-management mechanism because important session information can remain on the server. Cookies can be used to carry the session identifier and store non-sensitive preferences.

Question 3 – JDBC-Based E-Commerce Application

An e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

1. JDBC Architecture



Java Application: Sends database requests.

JDBC API: Provides Java interfaces for database connectivity.

DriverManager: Manages JDBC drivers and database connections.

JDBC Driver: Converts JDBC calls into database-specific operations.

Database: Stores and retrieves product information.

2. Product Database Table

```
CREATE DATABASE ecommerce;

USE ecommerce;

CREATE TABLE product (
id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100),
price DOUBLE,
quantity INT
);
```

3. Steps in JDBC Database Connectivity

Import JDBC packages.

Load/register the JDBC driver when required by the driver version.

Establish a connection using DriverManager.

Create a Statement or PreparedStatement.

Execute the SQL query.

Process the ResultSet for SELECT operations.

Close ResultSet, Statement/PreparedStatement, and Connection.

Handle SQLException using proper exception handling.

4. Java JDBC Program – Insert and Retrieve Product Data

```
import java.sql.*;

public class ProductJDBC {

static final String URL =
"jdbc:mysql://localhost:3306/ecommerce";
static final String USER = "root";
static final String PASSWORD = "root";

public static void main(String[] args) {

try {
// Establish connection
Connection con =
DriverManager.getConnection(URL, USER, PASSWORD);

System.out.println("Database Connected");

// Insert product
String insertSQL =
"INSERT INTO product(name, price, quantity) " +
"VALUES (?, ?, ?)";
```

```
PreparedStatement ps =
con.prepareStatement(insertSQL);

ps.setString(1, "Laptop");
ps.setDouble(2, 55000);
ps.setInt(3, 10);

int rows = ps.executeUpdate();

System.out.println(rows + " product inserted.");

// Retrieve products
String selectSQL =
"SELECT * FROM product";

Statement stmt = con.createStatement();

ResultSet rs =
stmt.executeQuery(selectSQL);

while (rs.next()) {

System.out.println("ID: " +
rs.getInt("id"));

System.out.println("Name: " +
rs.getString("name"));

System.out.println("Price: " +
rs.getDouble("price"));

System.out.println("Quantity: " +
rs.getInt("quantity"));

System.out.println("----------------");
}

// Close resources
rs.close();
stmt.close();
ps.close();
con.close();

} catch (SQLException e) {

System.out.println(
"Database Error: " + e.getMessage());
}
}
}
```

5. SQL Query for Inserting Product Data

```
INSERT INTO product(name, price, quantity)
VALUES (?, ?, ?);
```

PreparedStatement is used to supply the product name, price, and quantity values.

6. SQL Query for Retrieving Product Data

```
SELECT * FROM product;
```

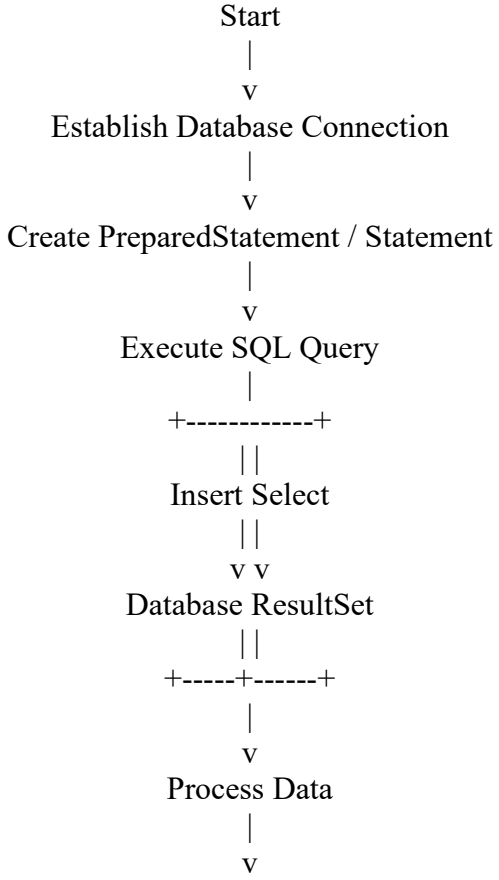
The ResultSet contains the rows returned by the SELECT query and can be processed using rs.next().

7. Exception Handling

```
try {
// JDBC operations
}
catch (SQLException e) {
System.out.println(
"Database Error: " + e.getMessage());
}
finally {
// Close resources if required
}
```

SQLException is handled so that database errors do not cause the application to terminate without a proper message.

8. JDBC Application Flow



Close Resources
|
v
End

Conclusion

JDBC provides connectivity between a Java application and a database. The e-commerce application uses PreparedStatement to insert product data and SELECT with ResultSet to retrieve product details. Proper exception handling and resource closing make the application safer and more reliable.